

• Medium #7, Reverse Integer

▲ Problem Statement → Given an signed 32-bit integer "x", return its digits reversed.

If the reverse number goes outside the 32-bit signed integer range :-

" $[-2^{31}, 2^{31} - 1]$ " return "0"

▲ Main Problem Solution → Extract the last digit of "x" using "% 10" & build the reversed no. digit by digit.

Before adding a new digit, check whether multiplying "rev" by "10" would cause an overflow.

This is important because the problem does not allow 64-bit ~~integers~~ integers.

▲ Brute Force Method: Convert the integer to a string, reverse the string, convert it back to an integer, & check for overflow.

Time: $O(\log x)$

Space: $O(\log x)$

▲ Algorithm: 1. Initialize "rev=0"

2. Extract the last digit:- "digit = $x \% 10$ "

3. Remove the last digit:- " $x = x / 10$ "

4. Check if adding the digit would overflow "int"

5. Add the digit to "rev"

6. Repeat until "x" becomes "0"

7. Return "rev"

8. If overflow is detected, return "0"

▲ Important Points: • " $\% 10$ " extracts the last digit

• " $/ 10$ " removes the last digit

• Negative numbers work automatically with C++ integer arithmetic

- "INT_MAX = 2147483647", So the last allowed digit is "7"
- "INT_MIN = -2147483647", So the last allowed digit is "-8"
- Overflow must be checked before $rev * 10 + digit$
- No "long long" is used

▲ Complexity → Time → $O(\log_{10} |x|)$
 Space → $O(1)$